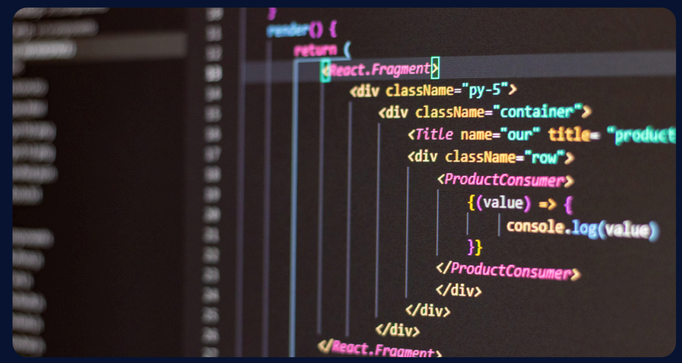


Library Management System - Case 6

- Library Management System
- Introduction to Programming 2 – Final Project
- Team members: Assel, Balnur, Aiym, Zhannur

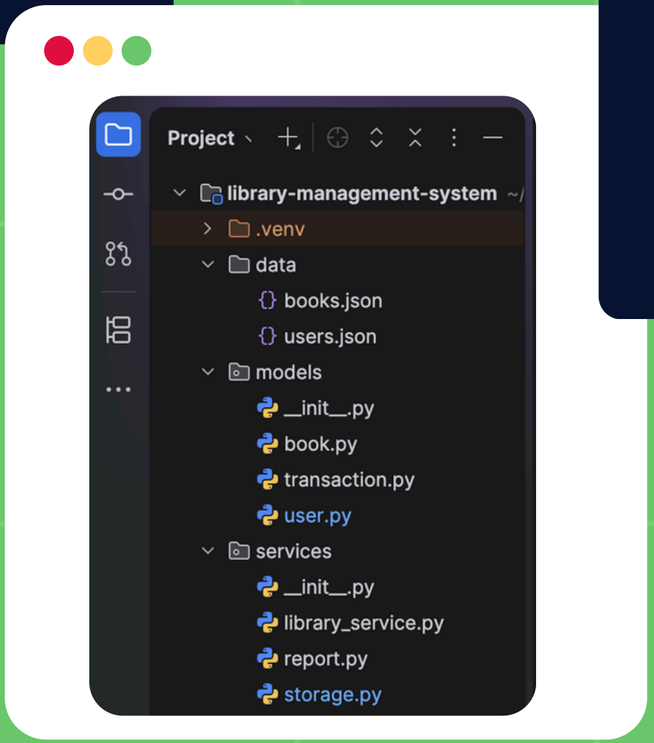
Problem description

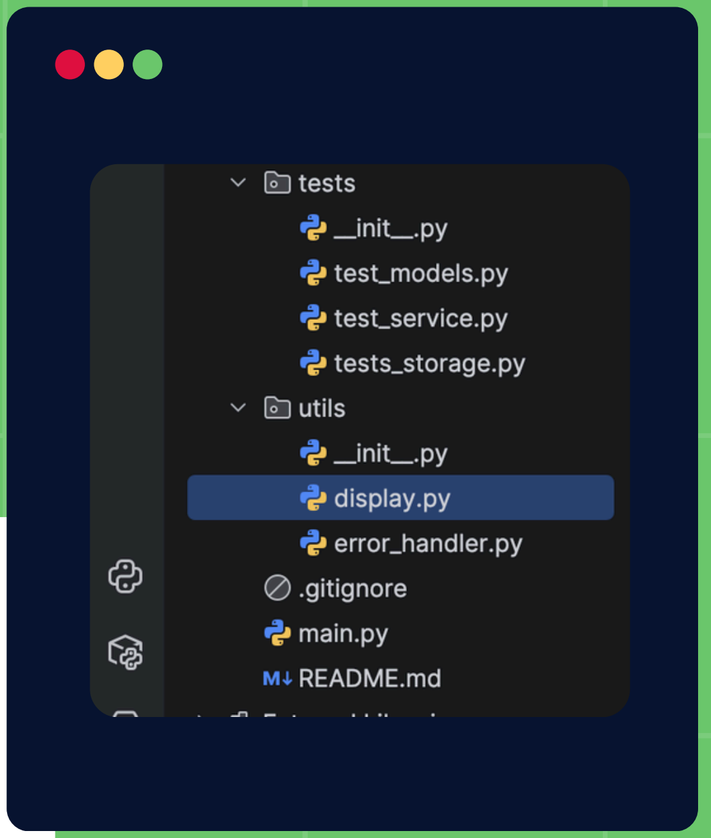
Case 6: Library Management System

- Problem: Libraries need to track books and users manually which is slow and error-prone.
- Solution: Our system automates:
  - Borrowing and returning books
  - Tracking availability
  - Maintaining user history
  - Generating reports

- 4 modules working together:
  - models/ — Book, User, Transaction classes
  - services/ — storage, library service, report
  - utils/ — display, error handling
  - data/ — books.json, users.json

System design





OOP Design

```
class User:
    def can_borrow(self):
        raise NotImplementedError

class Member(User):
    MAX_BOOKS = 3
    def can_borrow(self):
        return len(self._borrowed_books) <
self.MAX_BOOKS

class Admin(User):
    def can_borrow(self):
        return True # no limit
```

- 4 classes with inheritance
- Polymorphism: can_borrow() works differently for Member and Admin
- Encapsulation: private attributes with @property

Data Structures

- Dictionary - for books and users allows O(1) lookup
- Lists - for borrowed books and transactions
- Tuples - for immutable log entries
- Sets - in suggest_books() for fast intersection

- # BAD - O(n) loops through everything

```
for book in books:
    if book.id == book_id:
        return book
```

- # GOOD - O(1) instant lookup

```
book = self.books[book_id]
```

Key Features

- Borrow logic with 3 safety checks
- Return logic
- Reports: available books, borrowed books, user history

```
def borrow_book(self, user_id, book_id):
    if not book.available:
        raise ValueError("Not
available.")
    if not user.can_borrow():
        raise ValueError("Limit
reached.")
    if book_id in user.borrowed_books:
        raise ValueError("Already
borrowed.")
```

Extra Feature: suggest_book()

```
    #collect all books similar users borrowed
    suggestions = set()
    for similar in similar_users:
        suggestions |= set(similar.borrowed_books)

    #remove books this user already has
    suggestions -= user_books
    result = [self.books[book_id] for book_id in suggestions if book_id in self.books and self.books[book_id].available]
    return result
```

```
def suggest_book(self :{users, books} , user_id):  ajym
    #extra feature: recommend books based on similiar users
    if user_id not in self.users:
        raise ValueError("Invalid user_id.")

    user = self.users[user_id]
    user_books = set(user.borrowed_books)
    #find users who borrowed at least one same book
    #set intersection is faster than nested loops
    similar_users = [
        user for uid, in self.users.items()
        if uid != user_id and set (user.borrowed_books) & user_books
    ]
```

Recommends books based on what similar users borrowed
- Uses set intersection — faster than nested loops $O(n^2)$

TESTING

```python
def test_to_dict(self):  yerassel
    b = Book( book_id: 1,  title: "Clean Code"
    d = b.to_dict()
    self.assertEqual(d["id"],  second: 1)
    self.assertEqual(d["title"],  second:
    self.assertTrue(d["available"])
```

```python
def test_member_limit(self):  yerassel
    m = Member( user_id: 1,  name: "Alice")
    m.add_borrowed(1)
    m.add_borrowed(2)
    m.add_borrowed(3)
    self.assertFalse(m.can_borrow())  #since reached limit
```

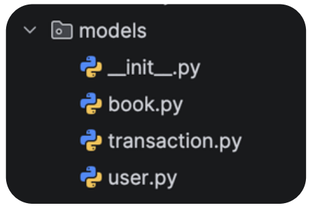
- Each person wrote unit tests for their own module
- Used unittest framework

```python
def test_borrow_unavailable_book_raises(self):
    with self.assertRaises(ValueError):
        self.service.borrow_book(3, 2)
```

Team Contribution

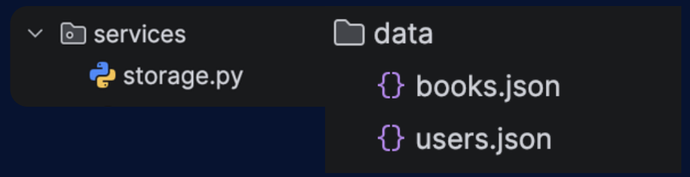
Person 1 - Assel

Data models & OOP



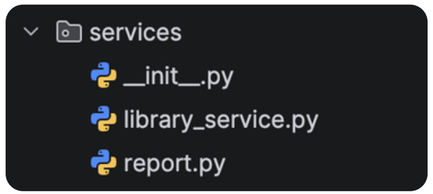
Person 2 - Balnur

File handling & storage



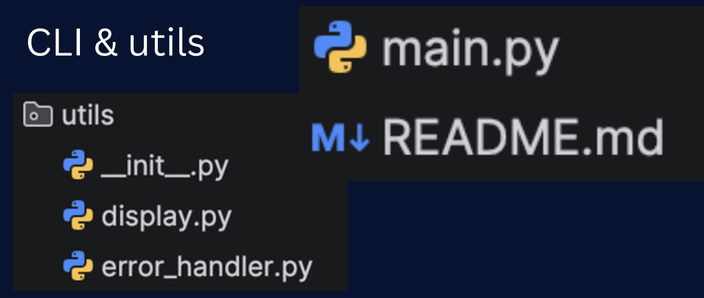
Person 3 - Aiym

Business logic & reports



Person 4 - Zhannur

CLI & utils



Conclusion

- Fully working Library Management System
- Covers all required concepts: OOP, collections, file handling, testing, advanced Python

- What we learned:
  - How to work as a team using Git
  - Why data structure choice matters
  - How to design modular code

Thank You